

Scenario

BloodBridge is a web portal maintained by a city hospital to connect voluntary blood donors with patients who suddenly need blood. Any citizen can register as a donor from the portal by giving their name, mobile number, blood group, district and the date on which they last donated blood. The hospital's duty officer then searches the register for a particular blood group in a particular district. The portal lists the matching donors with the longest gap since their last donation at the top, because those donors are the safest to call first. A donor who has given blood within the last 90 days is not allowed to register again, and once a donor has been called and has donated, the duty officer marks that donor as unavailable so that the same person is not called twice. The hospital has asked you to build this module using **procedural PHP** with a **MySQL** database, following the **MVC (Model-View-Controller)** architecture. Write your code in the exact files shown in the folder structure below and keep each layer's responsibility inside its own file.

Scenario Folder name → **bloodbridge**

```
├── model/db.php
├── model/DonorModel.php
├── controller/donor.php
└── view/donor.php
```

To Do Task	Marks
Model (Connection) File: model/db.php - Connect to the MySQL database bloodbridge_db on host localhost with the user root and an empty password using <code>mysqli_connect()</code>, and keep the returned link in a variable named <code>\$conn</code>. - If the connection fails, stop the script with <code>die()</code> and show the reason returned by <code>mysqli_connect_error()</code>. - Set the connection character set to utf8mb4 using <code>mysqli_set_charset()</code>. - This file must print nothing when the connection succeeds, so that the other files can safely load it with <code>require_once</code>. - The MySQL table donors already exists with the columns listed below – write all your queries against exactly these names: - donor_id – INT, PRIMARY KEY, AUTO_INCREMENT - full_name – VARCHAR(60) - mobile – VARCHAR(11) - blood_group – VARCHAR(3) - district – VARCHAR(40) - last_donation_date – DATE - is_available – TINYINT(1), default 1	7
Model (functionalities) File: model/DonorModel.php Write the four plain PHP functions given below – do not use a class. Every query must be executed as a prepared statement with <code>mysqli_prepare()</code>, <code>mysqli_stmt_bind_param()</code> and <code>mysqli_stmt_execute()</code>, the link <code>\$conn</code> must be received as the first parameter of every function, and no function may print anything. <code>saveDonor(\$conn, \$name, \$mobile, \$group, \$district, \$lastDate)</code> – runs an INSERT that stores one donor with <code>is_available</code> set to 1, and returns true or false depending on the result of <code>mysqli_stmt_execute()</code>. <code>searchDonors(\$conn, \$group, \$district)</code> – runs a SELECT for the available donors of the given blood group in the given district, ordered by <code>last_donation_date</code> ASC (oldest donation first), and returns the rows as an array built with a <code>mysqli_fetch_assoc()</code> loop. <code>countDonorsByGroup(\$conn, \$group)</code> – runs a SELECT COUNT(*) query and	

returns the total number of available donors of the given blood group as an integer. 4. markUnavailable(\$conn, \$donorId) – runs an UPDATE that sets is_available to 0 for the row whose donor_id matches, and returns true or false.	
Backend (Controller) File: controller/donor.php • Load both model files with require_once. This file only prepares data – it must not contain any HTML, and no MySQL function may be called here directly; every database operation has to go through the model functions. • When the registration form arrives by POST, trim every input and validate that no field is empty, that mobile is exactly 11 digits and starts with 01, that blood_group is one of A+, A-, B+, B-, O+, O-, AB+, AB-, and that last_donation_date is not a future date. • Apply the hospital rule: if the last donation date is less than 90 days before today, reject the registration with the message “You are not eligible to donate yet.” Otherwise call saveDonor() and store a success message. • When the search form arrives by GET with blood_group and district, call searchDonors() and countDonorsByGroup() and keep the returned data in variables for the view. • When the request carries action=unavailable together with a donor id, call markUnavailable() and then redirect back to the same page using header() so that refreshing the browser does not repeat the update. • Collect all error messages in an array and load the view at the end of the file with include.	8
Frontend (View) File: view/donor.php • A Donor Registration form that submits to the controller by POST with Full Name, Mobile, Blood Group (a <select> holding the eight groups), District and Last Donation Date (<input type="date">). • A Search Donor form that submits to the controller by GET with a blood group dropdown and a district field. • Show the success message in green and every validation error in red, and keep the values the user already typed inside the fields after a failed submission. • Show the search result in an HTML table with the columns SL, Full Name, Mobile, Blood Group, District, Days Since Last Donation, and a “Mark Unavailable” link on every row. Use a foreach loop over the array the controller has already prepared; no MySQL function may be called in this file. • Print the total number of matching donors above the table, print “No donor found.” when the result is empty, and escape every printed value with htmlspecialchars().	5